3. Hussain - Cryptography Engine And Security Logic

Main Responsibilities

Hussain handled the cryptographic implementation and security mechanisms.

His areas:

- AES encryption/decryption.
- RSA encryption/decryption.
- Digital signatures.
- SHA-256 hashing.
- Replay fingerprinting.
- Password hash verification support.
- Crypto API routes.

Modules Owned

- `backend/app/crypto_engine/aes_service.py`
- `backend/app/crypto_engine/rsa_service.py`
- `backend/app/crypto_engine/signature_service.py`
- `backend/app/crypto_engine/hash_service.py`
- `backend/app/routes/crypto.py`

Work Details

Hussain implemented the cryptographic services that support the educational demonstrations.

AES-256-GCM is used to turn readable packets into ciphertext. RSA and digital signatures are available through the crypto API. SHA-256 is used for hashing and replay request fingerprinting.

How He Should Explain His Work

I worked on the cryptography engine. I implemented AES-256-GCM for symmetric encryption, RSA-OAEP for asymmetric encryption, RSA-PSS for digital signatures, and SHA-256 hashing. These concepts are used to show how encryption protects intercepted packets, how hashes detect duplicate requests, and how digital signatures provide integrity and authenticity.

Key Points To Mention

- AES protects packet contents.
- RSA demonstrates public/private key cryptography.

- Digital signatures verify message authenticity.
- SHA-256 creates fixed fingerprints.
- Crypto routes allow testing each cryptographic function.

Hussain Presentation Guide

What Hussain Should Say

My role was the cryptography engine. I implemented AES-256-GCM, RSA-OAEP, digital signatures using RSA-PSS, and SHA-256 hashing. These cryptographic services support the project scenarios and demonstrate confidentiality, integrity, authentication, and secure communication concepts.

Work Area Explanation

Hussain worked on cryptographic services.

How it was implemented:

1. AES service encrypts and decrypts text using AES-256-GCM.
2. RSA service generates public/private key pairs.
3. RSA encryption uses OAEP with SHA-256.
4. Signature service signs and verifies messages.
5. Hash service creates SHA-256 digests.
6. Crypto routes expose these services through API endpoints.

Modules To Mention

- `aes_service.py`
- `rsa_service.py`
- `signature_service.py`
- `hash_service.py`
- `encoding.py`
- `routes/crypto.py`

Viva Questions For Hussain

Q: What is AES used for in this project?

A: AES encrypts sensitive payloads so intercepted data becomes unreadable ciphertext.

Q: Why AES-256-GCM?

A: AES-256 is strong, and GCM provides both confidentiality and integrity.

Q: What is RSA used for?

A: RSA demonstrates public/private key encryption and secure key exchange concepts.

Q: What is a digital signature?

A: It proves message authenticity and detects tampering.

Q: Where is SHA-256 used?

A: It is used in the crypto API and for replay request fingerprinting.